

NAME(S):

The Straight and Narrow

Assignment Set-up

- start as an in-class activity, working in pairs
- finish independently as part of HW 4, due 4/29/26
- bring written answers to class for 0.5 engagement points
- submit code on gradescope for corrections

Objectives

- To practice with all the structures we've learned up to this point:
 - variables, strings, functions (both user-defined and built-in), conditionals, and loops
- To see first-hand the problem of intractability
- To think further about algorithm design

Reminders for Pair Programming

Pair programming guidelines:

- Roles:
 - driver: has the keyboard and types
 - navigator: guides the driver
- Instructions:
 - You should be talking to each other constantly!
 - You should be able to swap roles at any moment.
 - Swap roles every 5 to 10 minutes.

Group work preparation

- after reading through the entire assignment, complete this section before any other

1. What knowledge and or skills will you need to complete this assignment?

2. Do you know where can you gain those, independently, before beginning work? If so, what resources will you use?

3. What will you be relying on from your partner(s)? What knowledge and or skills will your partner(s) need to complete this assignment?

5. What resources (could be from the course website or from anywhere else) can you suggest they use to gain that knowledge and skills independently?

6. What resources did they suggest for you?

Introduction

The title of this project comes from the fact that while a straightforward way of designing an algorithm is often a good place to start, one should not limit oneself to only considering such a narrow set of solutions. In fact, writing a computer program that solves a problem in a straightforward way often leads to intractable solutions: programs that cannot give you an answer in a reasonable amount of time. In this project, you will see that first-hand as you build a small-scale version of BLAST – the sequencing alignment software – and see how a clear-cut approach to alignment results in a system that cannot handle any but the smallest of inputs.

How BLAST works

The Basic Local Alignment Search Tool (BLAST) does an alignment match between a given, often unknown, sequence and the sequences in its database. It then shows you best matches so that you know what your unknown sequence is related to. For example, if you give BLAST the amino acid sequence SGYR, it may produce the alignment:

```
SG-YR
SRSYR
```

where the top sequence is your given sequence and the bottom one is from its database. BLAST works by padding “blanks” to the shorter sequence to indicate places where mutations occurred; that is, where insertions or deletions of sets of amino acids took place. A pairwise alignment of each position is then done, with points being given for a matching position and fewer points given (or even points taken away!)

for a mismatching position. For example, we might give 1 point for an exact match and 0 points for a mismatch. For the example above, we would get a final score of 3 points: 1 each for the first column (S), the fourth column (Y), and the fifth column (R). The mismatch (G with R) or matches involving a blank would not score any points.

BLAST does this repeatedly with every sequence in its database, and whichever alignment gives the best score is the one that would appear as the first hit on a BLAST results page. BLAST is a huge piece of work, and we could never hope to duplicate it all in a week (or even ten). So we make the following simplifying assumptions. First, your program will only be comparing two protein sequences at a time (instead of one input sequence being compared to an entire database of data). Second, one of the two sequences will be exactly three amino acids long. This isn't realistic, of course, but as you'll see, even such a small input sequence will be able to wreak havoc later.

Part 1: Build some useful Helper Functions

To make this project manageable, let's first build three smaller functions that can make the bigger task of alignment a lot easier.

1. Make a `pad_with_gaps` function with the following definition line:

```
def pad_with_gaps(seq):
```

This function takes a protein sequence as a parameter. It will then create and return another string that has the same length as `seq`, but is made up entirely of blanks (dashes). For example, if the length of `seq` is 5, this function would return `"-----"`. Later on, you'll use this function to take the longer of the two input sequences and make a "blanks" version of it.

2. Make a function `replace` with the following definition line:

```
def replace(someString, position, letter):
```

This function will create a new string identical to `someString` except that the character at `position` will be replaced with `letter`. For example, if you called this function with: `replace("-----", 1, "V")` it would return the string `"-V----`". Hint: you can build this by concatenating together the part of `someString` that comes before `position`, then the letter, and finally, the part of `someString` that comes after `position`.

3. Make a function `find_score` with the following definition line:

```
def find_score(first,second):
```

This function will take as inputs two equal-length strings: `first` and `second`. It will do a pairwise comparison of each position and return the alignment score. When there is a match, the score should increase by 1. A mismatch will leave the score the same. For example, calling this function with: `find_score("MPFVS", "MS-V-")` would return a score of 2. Be sure to run some tests with each of these functions before moving on. Keep those tests in your code so I can see that you've done them.

Part 2: BLAST off

It's now time to write the alignment matcher. The straightforward algorithm we'll be using is to try every possible combination of where the "blanks" could be in the shorter sequence. We'll find all the scores of all of the possibilities, and the biggest one will be our best hit. For example, if the two protein sequences to align were "MPFVS" and "MSV", here's how the alignment matcher will use the helper functions you just wrote to get the job done. Each row in the table below represents one possible alignment.

Use pad_with_gaps to make:	Use replace to make:	Use find_score to align:	find_score will return:
"-----"	"MSV--"	"MPFVS" "MSV--"	1
"-----"	"MS-V-"	"MPFVS" "MS-V-"	2
"-----"	"MS--V"	"MPFVS" "MS--V"	1
"-----"	"M-SV-"	"MPFVS" "M-SV-"	2
"-----"	"M-S-V"	"MPFVS" "M-S-V"	1
"-----"	"M--SV"	"MPFVS" "M--SV"	1
"-----"	"-MSV-"	"MPFVS" "-MSV-"	1
"-----"	"-MS-V"	"MPFVS" "-MS-V"	0
"-----"	"-M-SV"	"MPFVS" "-M-SV"	0
"-----"	"--MSV"	"MPFVS" "--MSV"	0

So the second column in the table above represents every possible position that M, S, and V could have when aligning with the "MPFVS" sequence. Your final job is to write this function:

```
def find_alignment(shorter, longer):
```

where `shorter` is the three-amino-acid sequence and `longer` is any length sequence greater than three. This function will not return anything. It will simply find the best alignment and print it along with the alignment score. So for the example above, this function would print

```
MPFVS
MS-V-
Score = 2
```

which corresponds to the second row in the table above. (Printing the alignment in the fourth row would also be ok. You need only print one best alignment.) Make sure you print it just like the example above! You want an alignment – just like BLAST gives. You will need to use three nested for loops to get the

computer to process all of the different combinations of where the three amino acids in the shorter sequence could be placed in the longer sequence of blanks. Here is the pseudocode of what your function should do:

```
for every position (call it i) that the first aa can have:
    for every position (call it j) that the second aa can have
        (to the right of the first aa):
            for every position (call it k) that the third aa can have
                (to the right of the second aa):
                    make a string of blanks (with length of the longer sequence)
                    put the first aa into position i
                    put the second aa into position j
                    put the third aa into position k
                    find the score of that alignment
                    if that alignment has the best score seen so far
                        save the score and the alignment
print the score and the alignment
```

Testing

Once your function is working, test it by giving it two protein sequences to align. Remember, the shorter one must be three amino acids long. The longer one can be any length, but you should notice pretty quickly that the longer you make it, the slower the program will run. Time how long the program takes for different length sequences. Start at length 10 for the longer sequence, and increase it by ten each time until the wait time becomes more than one minute. Make a table of your measurements (length of longer sequence vs. completion time) and include it with your project as comments.

As a final test, take a real protein like EMSY from Activity 1 and align that 1322-aa sequence against any 3-aa sequence. You may want to have a book to read or go get a cup of coffee once you start it running. It will actually take days to finish, so just quit IDLE once you've let it run a few minutes.

To be continued...

Hopefully, your tests have convinced you that while this "brute force" way of doing alignment works, it does not scale very well, even when one of the sequences is just three amino acids long! So how does BLAST do it? We need smarter algorithms. In your next project, which will be a group project with a CS team member, you'll help implement and test a more sophisticated algorithm to avoid the intractability problem.

How to turn this in

Please name your file as "first_blast.py". Include comments indicating how specifically each group member contributed. If you finish in class you may submit together but if you finish the activity individually, you should each submit your final versions. Submit your code on Gradescope for corrections. Bring your written answers to class for 0.5 engagement points.